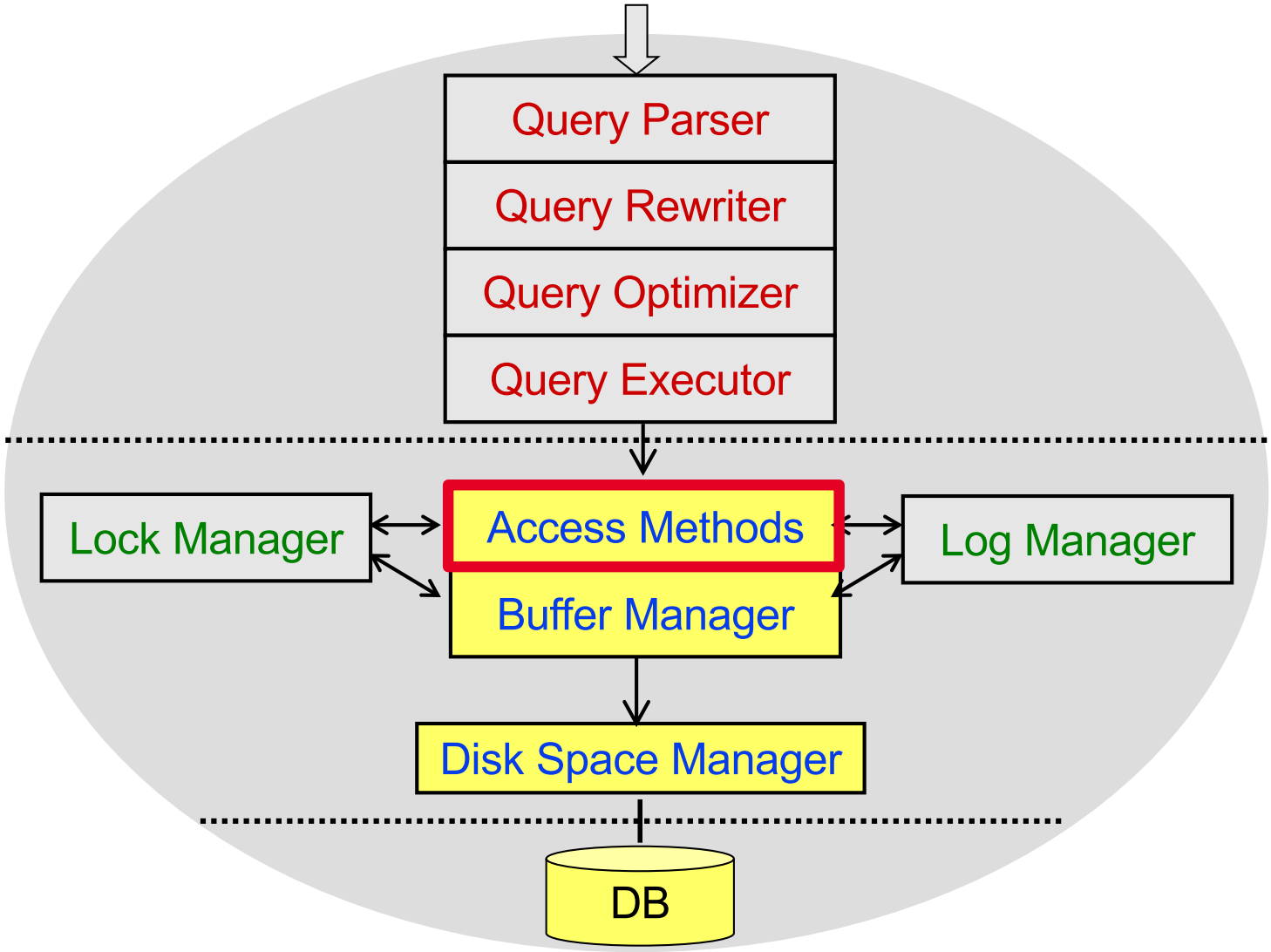
Database Indexing Beyond B+ Trees

Ioana Manolescu
Inria and Ecole Polytechnique



Slides Courtesy of R. Ramakrishnan, J. Gehrke, Y. Diao

DBMS Architecture



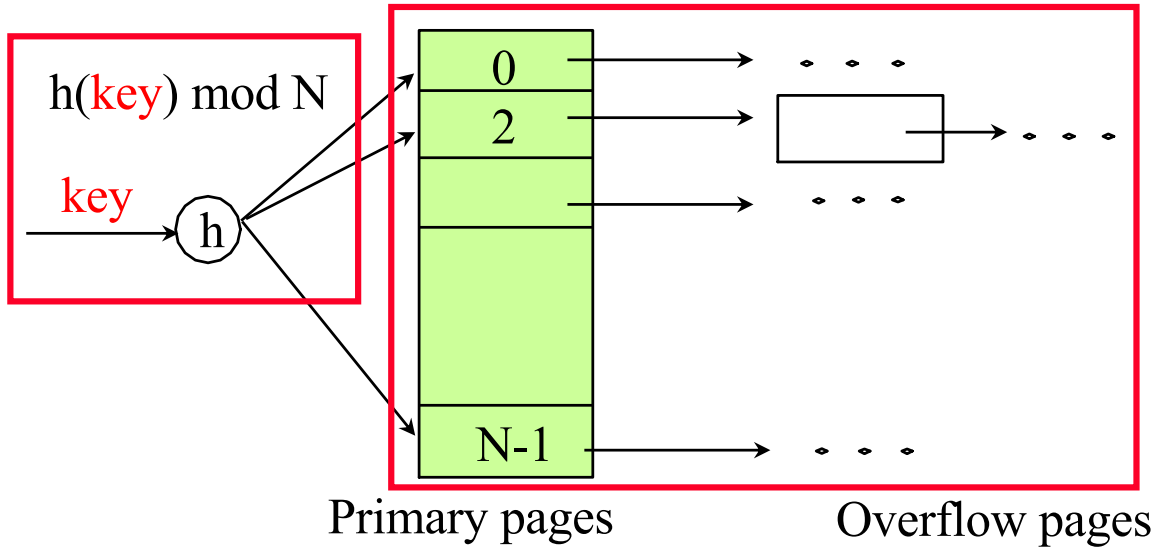
Outline

- **Hash-based** indexes
- Indexing for **full-text** search
- R-Tree: **spatial (multidimensional) indexing**
- **Multidimensional indexing** for nearest-neighbor search

Hash-based indexing

- *Hash-based* indexes can be used for *equality search*.
  - E.g., retrieve a student **with id '123'**
- No support for order-aware search
  - Not possible to find all students whose age is >20
- As for any index, 3 alternatives for data entries **k***:
  - **<k**, data record with key value **k>**
  - **<k**, rid of data record with search key value **k>**
  - **<k**, list of rids of data records with search key **k>**
- We will see:
  - **Static** hashing
  - **Dynamic** hashing

Static Hashing



- **h**(*k*) mod N = bucket to which data entry with key *k* belongs. k1≠k2 can lead to the same bucket.
- Static structure: # buckets (N) fixed
  - *Primary pages*: allocated sequentially, never de-allocated;
  - *Overflow pages*: allocated/de-allocated if needed.

Static Hashing (Contd.)

- Hash fn on the *search key* distributes values over [0 ... N-1].
  - **h**(*key*) mod N = (a * *key* + b) mod N
  - a and b are constants; a lot is known about how to tune **h**

- Buckets contain data entries in a chain of pages.
  - Long overflow chains degrade performance.
  - **Dynamic** techniques fix this problem (see next):
    - Extendible Hashing
    - Linear Hashing

Extendible Hashing

- When bucket (primary page) becomes full, re-organize file by ***doubling*** number of buckets
  - But: reading and writing all pages is expensive

- **Idea**: use a *directory of buckets*. When bucket is full:
  1) *double the directory,*
  2) *split just the bucket that overflowed.*

  - Directory much smaller than file, so doubling is cheap.
  - Only one page of data entries is split. *No overflow page*!
  - Smart adjustment of the hash function.

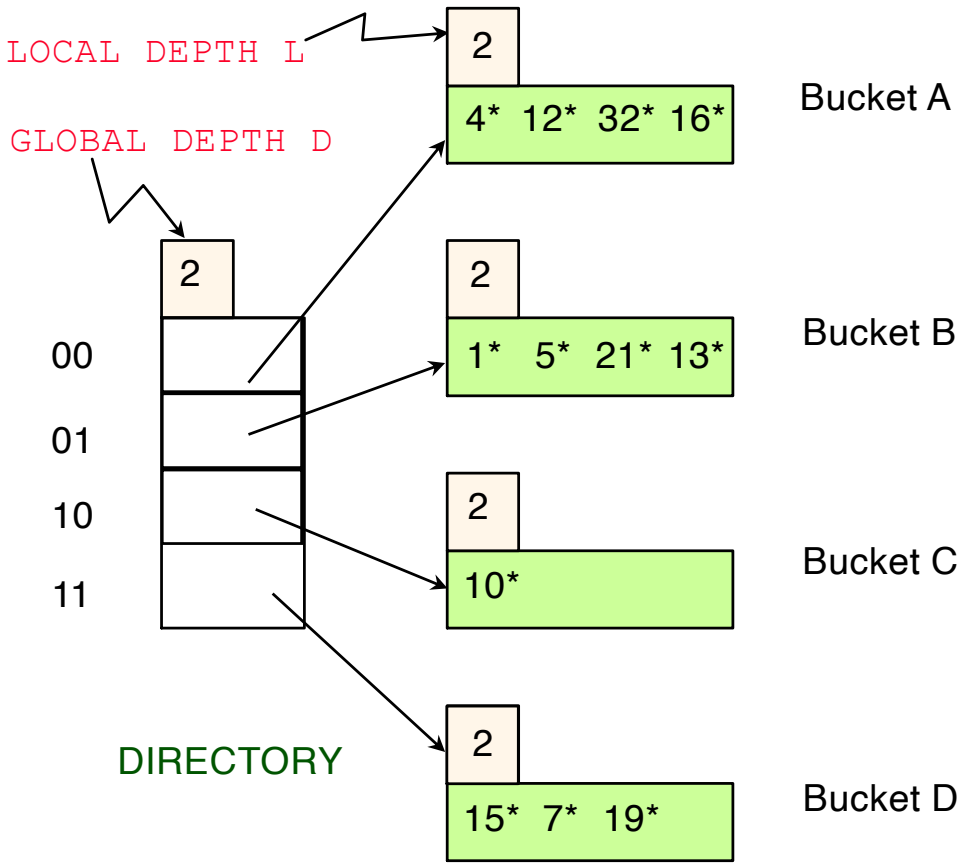
Example

- Directory is array of size N=4, *global depth* D = 2.

- To find bucket for *key*:
  1) get **h**(*key*),
  2) take last '*global depth*' # bits of **h**(*key*), i.e., mod 2^D.
     - If **h**(*key*) = 5 = binary 101,

  take last 2 bits, go to bucket pointed to by 01.

- Each bucket has *local depth* L (L ≤ D) for splits!

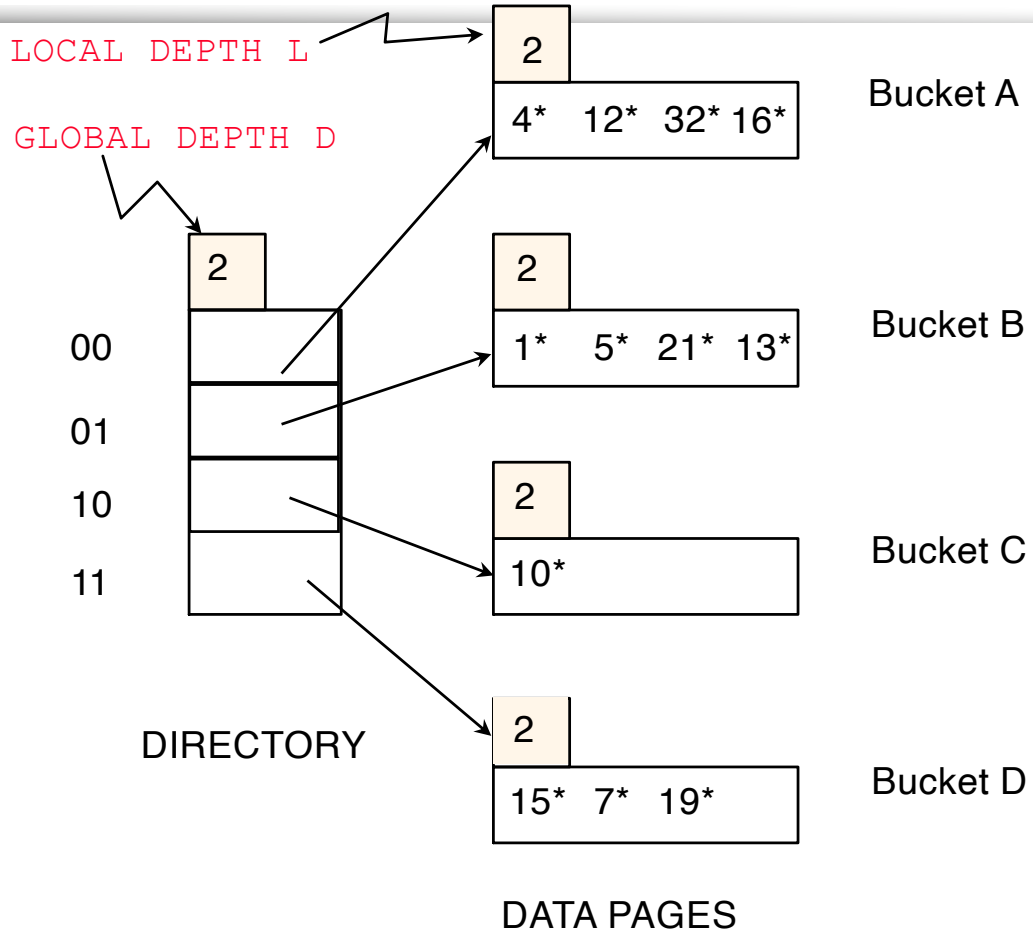


Inserts

- If bucket is full, *split* it:
  - *Allocate new page,*
  - *Re-distribute,*
  - If needed, *double* directory.

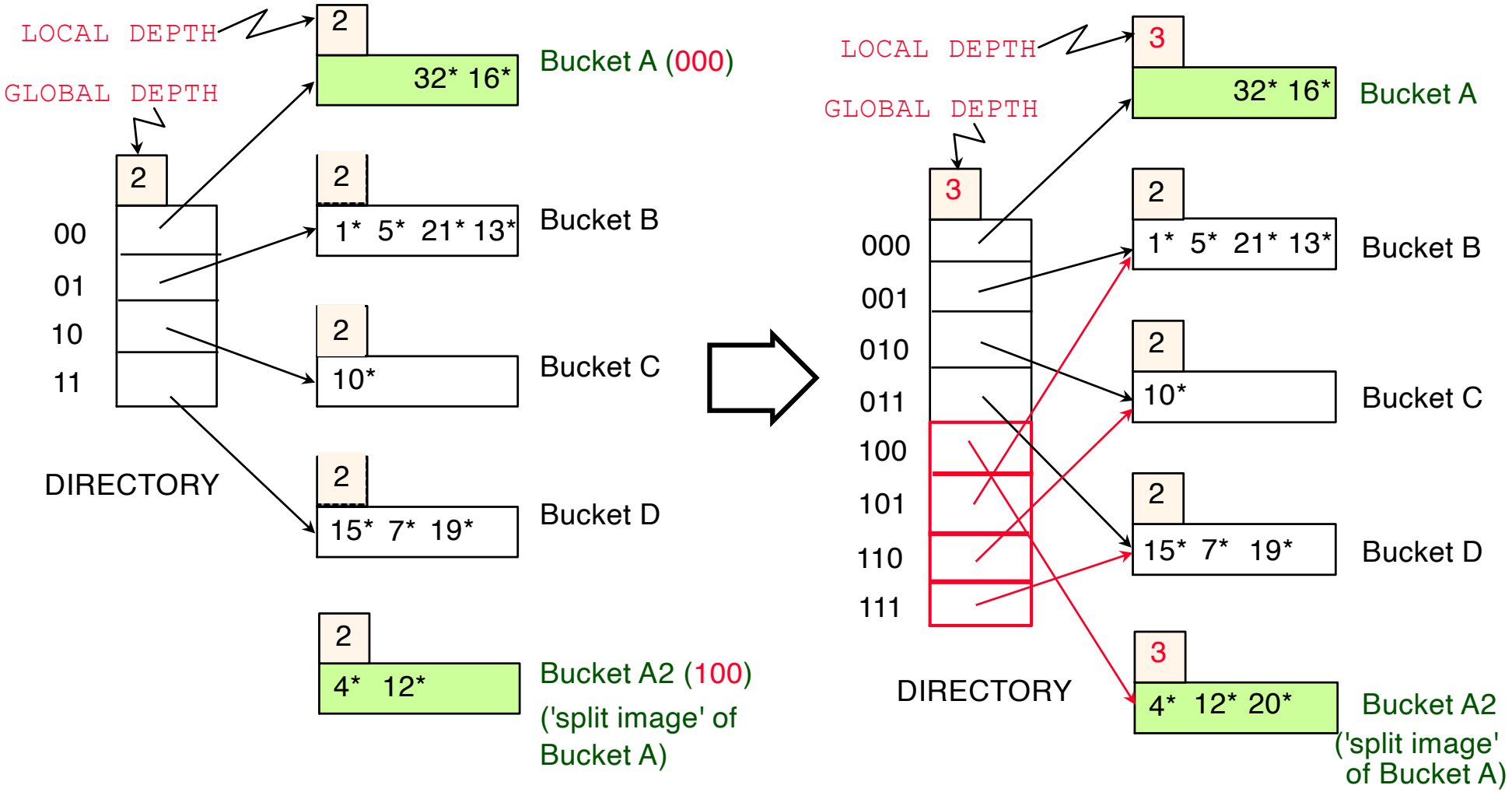
- Double the directory if *global depth D = local depth L*
  - Split if D = L.
  - Otherwise, don't

(see next)



Insert k^* with h(k)=20?

Insert h(k)=20 (Causes Doubling)



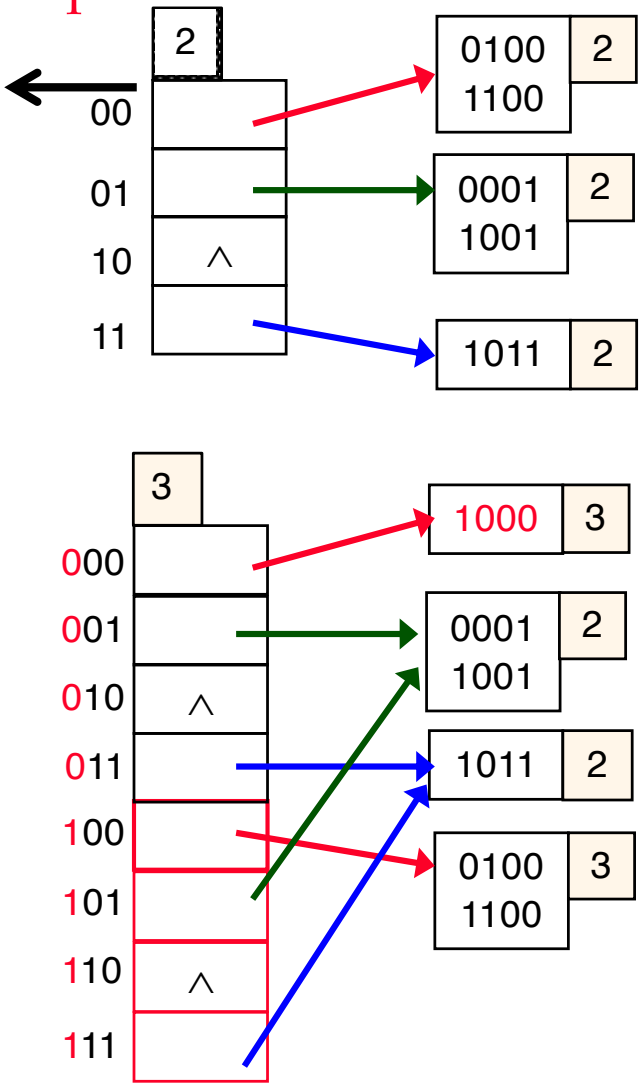
Points to Note

- 20 = binary 10100. Last **3** bits needed to distinguish A, A2.
  - *Global depth D of directory*: **Max #** of bits needed to tell which bucket an entry belongs to.
  - *Local depth L of a bucket*: **Actual #** of bits needed to determine if an entry belongs to **this bucket**.

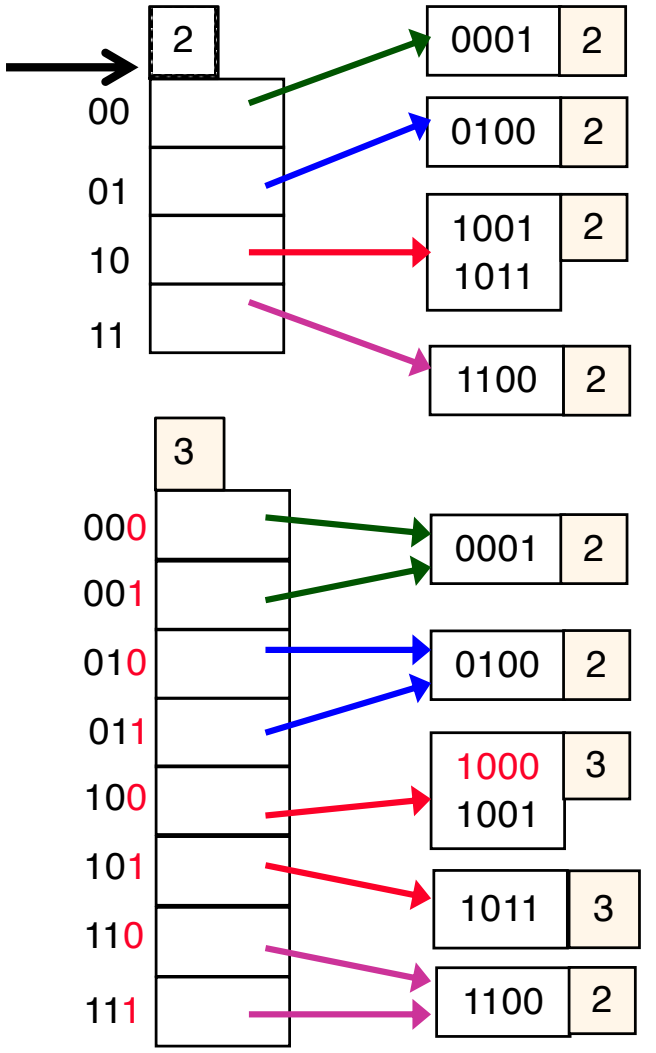
- Bucket split causes directory doubling if before insertion, *L* of bucket = *D of directory*.

Directory Doubling (inserting 8)*

Using least significant bits, split adds to the left



Using most significant bits, split adds to the right →



cannot reuse non-split part of directory

Deletes

- Remove a data entry from bucket
  - If bucket is empty, can be merged with 'split image'.
  - If each directory entry points to same bucket as its split image, can halve directory.
  - If assume more inserts than deletes, do nothing…

Comments on Extendible Hashing

- *Access cost*: If directory fits in memory, equality search takes **one I/O** to access the bucket; else **two**.
- *Skews*: If the distribution of *hash values* is skewed, directory can grow large.
- *Duplicates*: Entries with *same key value* need overflow pages!

Linear Hashing

- Also dynamic hashing; an alternative to Extendible Hashing.
- Advantage: adjusts to inserts/deletes *without a directory*.

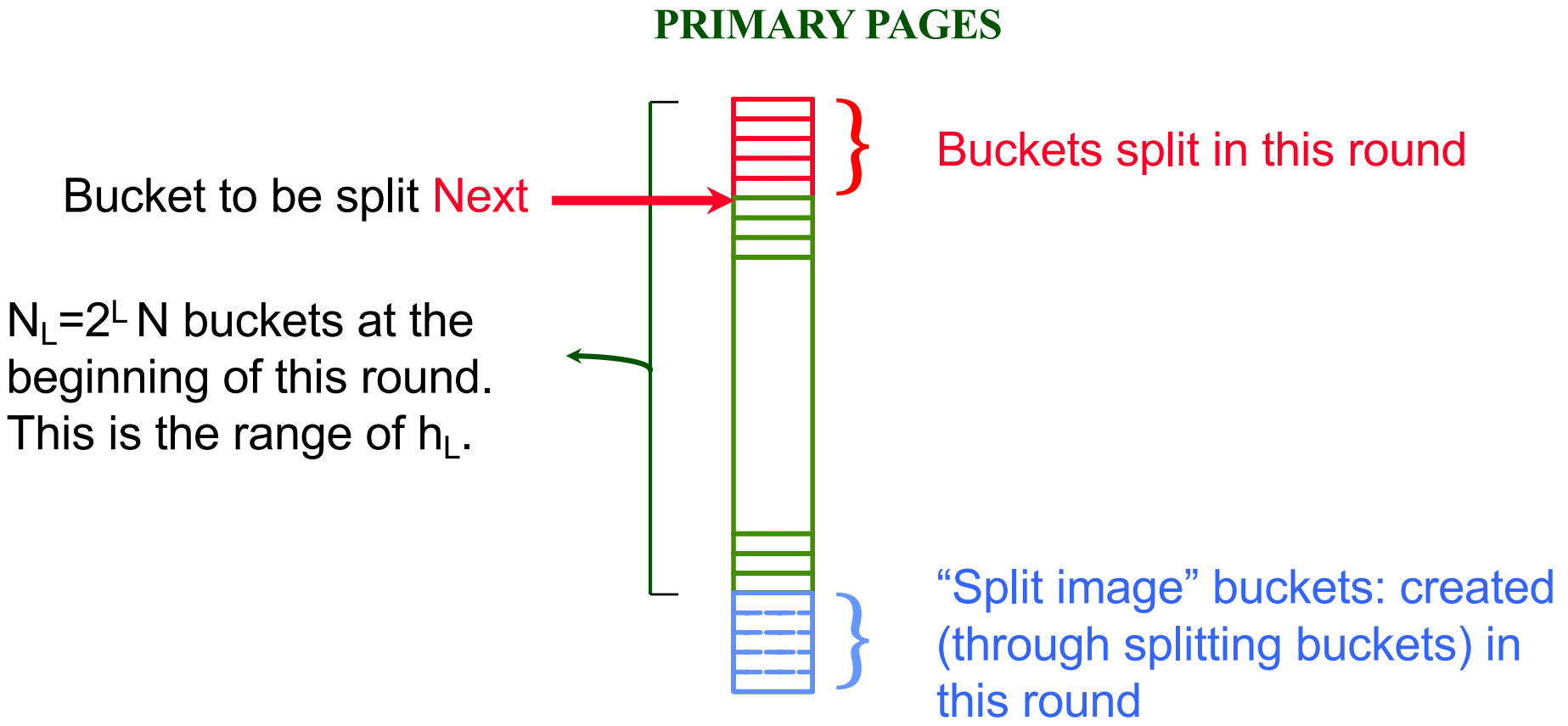
- **Idea**: Use a family of hash functions $\mathbf{h}_0$, $\mathbf{h}_1$, $\mathbf{h}_2$, ...
  - $\mathbf{h}_i(key) = \mathbf{h}(key) \bmod (2^i * N)$; N = initial # buckets
  - **h** is some hash function (range is *not* 0 to N-1)
  - $\mathbf{h}_0 = \mathbf{h}(key) \bmod N$
  - $\mathbf{h}_{i+1}$ doubles the range of $\mathbf{h}_i$ (similar to directory doubling)
  - If $N = 2^{d0}$ for some *d0*, $\mathbf{h}_i$ consists of applying **h** and looking at the last $di = d0 + i$ bits.

Linear Hashing (Contd.)

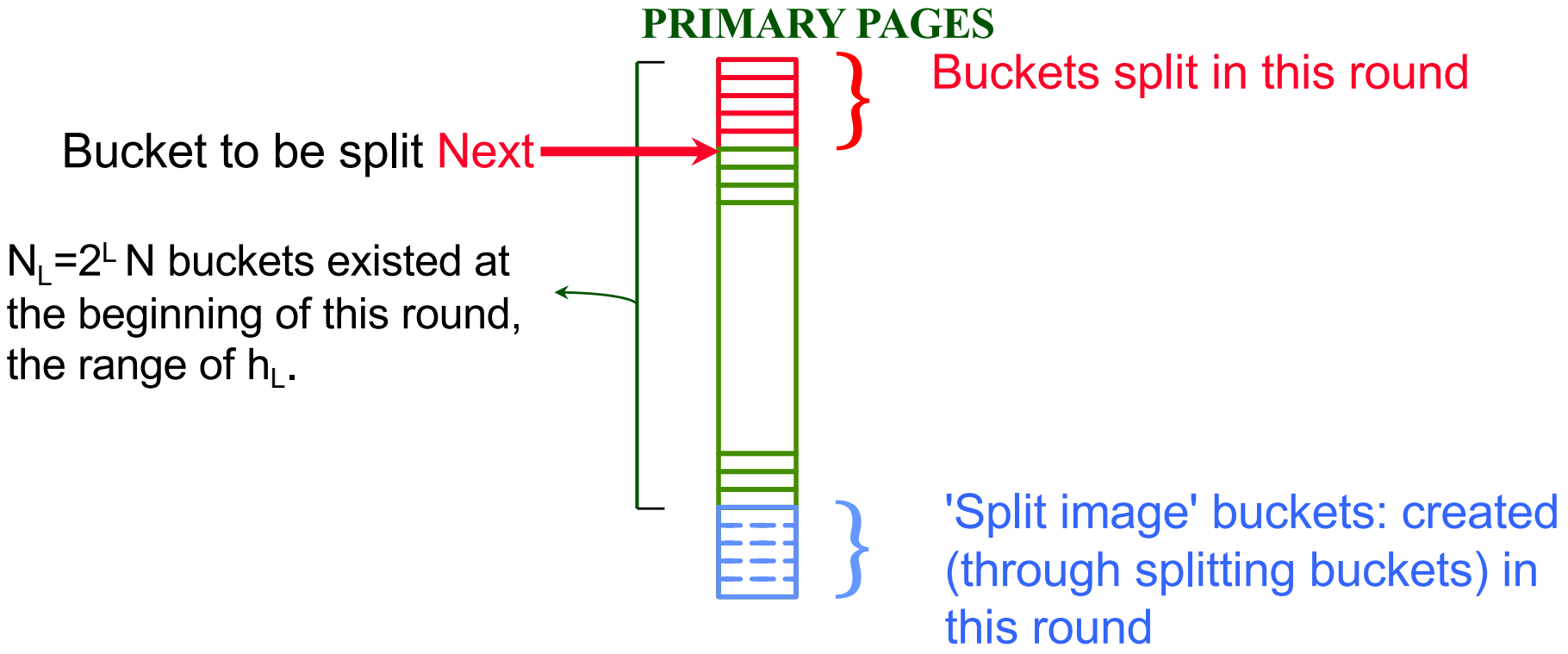
- LH avoids directory by
  1) using *temporary* overflow pages, and
  2) choosing bucket to split in a *round-robin* fashion.

- Splitting *(and the "life of the hash table")* proceeds in '*rounds*'.
  - Many successive operations may happen during one round!
- Round L starts with $N_L = 2^L N$ buckets:
  - *Next* bucket to be split: Buckets [0, *Next-1*] have been split; [*Next*, N_L] yet to be split.
  - Round L ends when all $N_L = 2^L N$ initial buckets have been split, thus there are $2^{L+1} N$ buckets

L^{th} Round of splitting



Searches

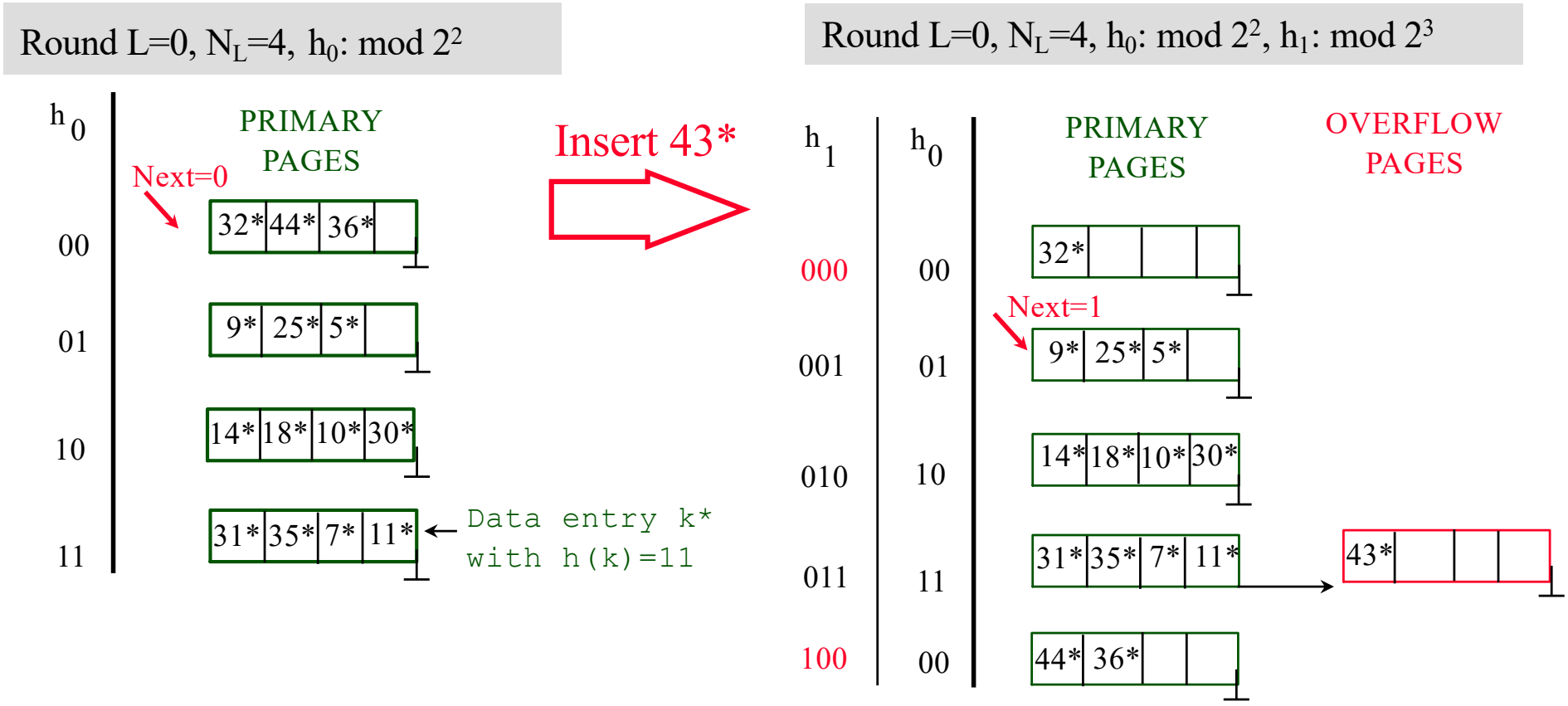


- **Search:** To find bucket for data entry k^*, apply $\mathbf{h}_L(k)$:
  - If $\mathbf{h}_L(k)$ in [*Next*, N_L], k^* belongs here.
  - Else, apply $\mathbf{h}_{L+1}(k)$ to choose between bucket $\mathbf{h}_L(k)$ and its split image bucket: $\mathbf{h}_L(k) + N_L$.

Inserts

- **Insert**: Find bucket *B* by applying $\mathbf{h}_L$ / $\mathbf{h}_{L+1}$. If B is full:
  - Add overflow page, insert data entry *k**.
  - (*Maybe*) split bucket *Next* (often $B \neq Next$) using $\mathbf{h}_{L+1}$, increment *Next*.
  - Can 'trigger' split when: overflow; or, when overflow and fill factor > x%...
- Since buckets are split round-robin, long overflow chains don't develop *(eventually all pages are split)*
- Compared to *Extendible Hashing*:
  - Switching of hash functions is *implicit* in how the # of bits examined is increased.
  - No need to *physically double the directory* (no directory)

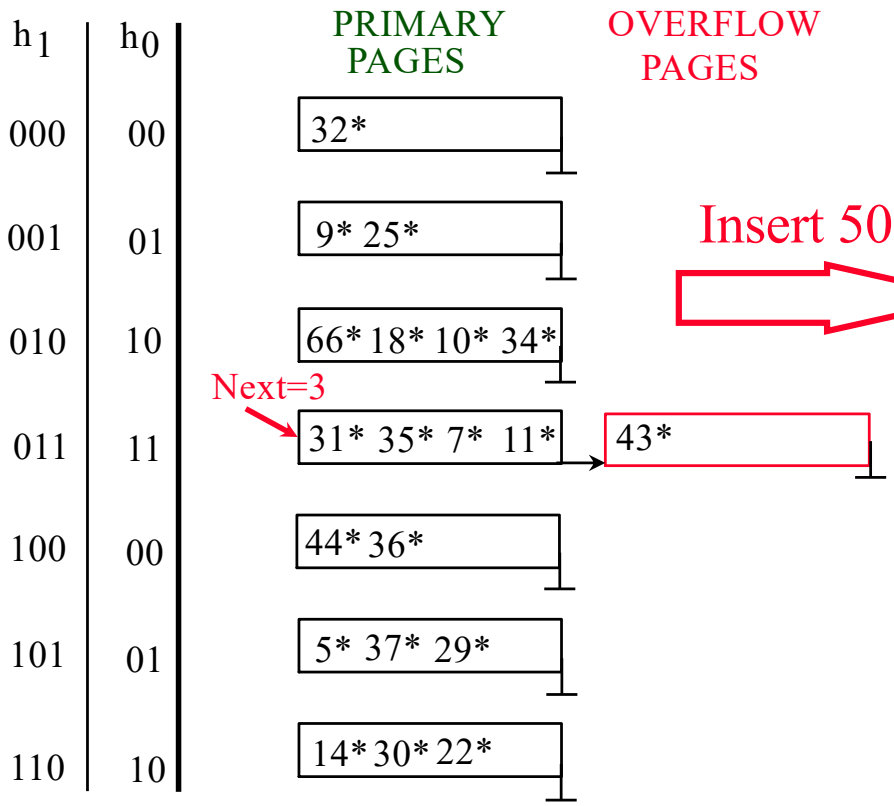
Example of Linear Hashing



- On split, $\mathbf{h_{L+1}}$ is used to *re-distribute* entries.

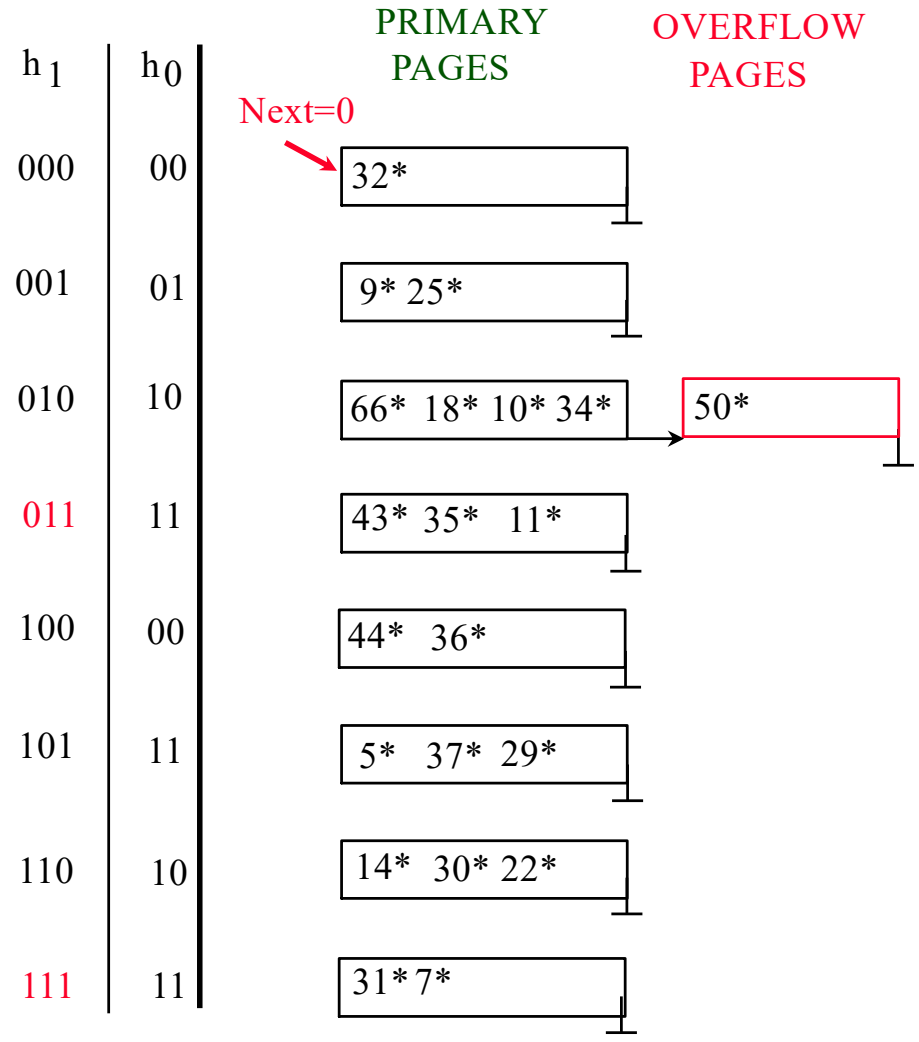
Example: End of a Round

Round L=0, N_L=4, h_0: mod 2^2, h_1: mod 2^3



Insert 50*

Round L=1, N_L=8, h_1: mod 2^3



End of round L when Next=N_L-1

Summary of Dynamic Hashing

- Linear Hashing (LH) avoids directory by splitting buckets round-robin, and using overflow pages.
  - Overflow chains not likely to be long.
  - Duplicates handled easily.
  - Space utilization can be lower than Extendible Hashing (EH), since splits not concentrated on "dense" data areas.

- *Skewed* data distributions: *hash values* of data entries are not uniformly distributed! Cause problems with EH and LH.

- PostgreSQL hash indexes: https://github.com/postgres/postgres/blob/master/src/backend/access/hash/README

Outline

- **Hash-based** indexes
- Indexing for **full-text** search
- R-Tree: **spatial (multidimensional) indexing**
- **Multidimensional indexing** for nearest-neighbor search

Indexing text for pattern-based search

- Frequent need when not sure exactly of a search string value and we accept approximate results
- At the **storage (index)** level:
  - Postgres data type: **ts_vector** (text search vector)
  - Functions to convert strings into ts_vector
  - select to_tsvector('english', 'Shall I compare thee to a summer day ?');

to_tsvector
'compar':3 'day':8 'shall':1 'summer':7 'thee':4

  - Words have been *stemmed*
  - *Stop words* have been removed
  - Words are *alphabetically sorted* and their *position* in the original string is recorded

Indexing text for pattern-based search

- Frequent need when not sure exactly of a search string value and we accept approximate results
- At the **storage (index)** level:
  - Postgres data type: **ts_vector** (text search vector)
  - Functions to convert strings into ts_vector
  - select to_tsvector('english', 'Shall I compare thee to a summer day ?');

to_tsvector
'compar':3 'day':8 'shall':1 'summer':7 'thee':4

  - create index ft_movie_title on movie using gin (to_tsvector('english', title));
  - select * from movie m
    where to_tsvector('english', title) @@ phraseto_tsquery('english', 'summer days');

Outline

- **Hash-based** indexes
- Indexing for **full-text** search
- R-Tree: **spatial (multidimensional) indexing**
- **Multidimensional indexing** for nearest-neighbor search

Spatial Indexing

- Applications: when deal with *n-dimensional objects*
  - Geographic Information Systems (GIS)
  - Mobile Databases
  - Magnetic Resonance Imagery (e.g. brain scan)
  - In comparison: B+-trees can be used along 1 dimension

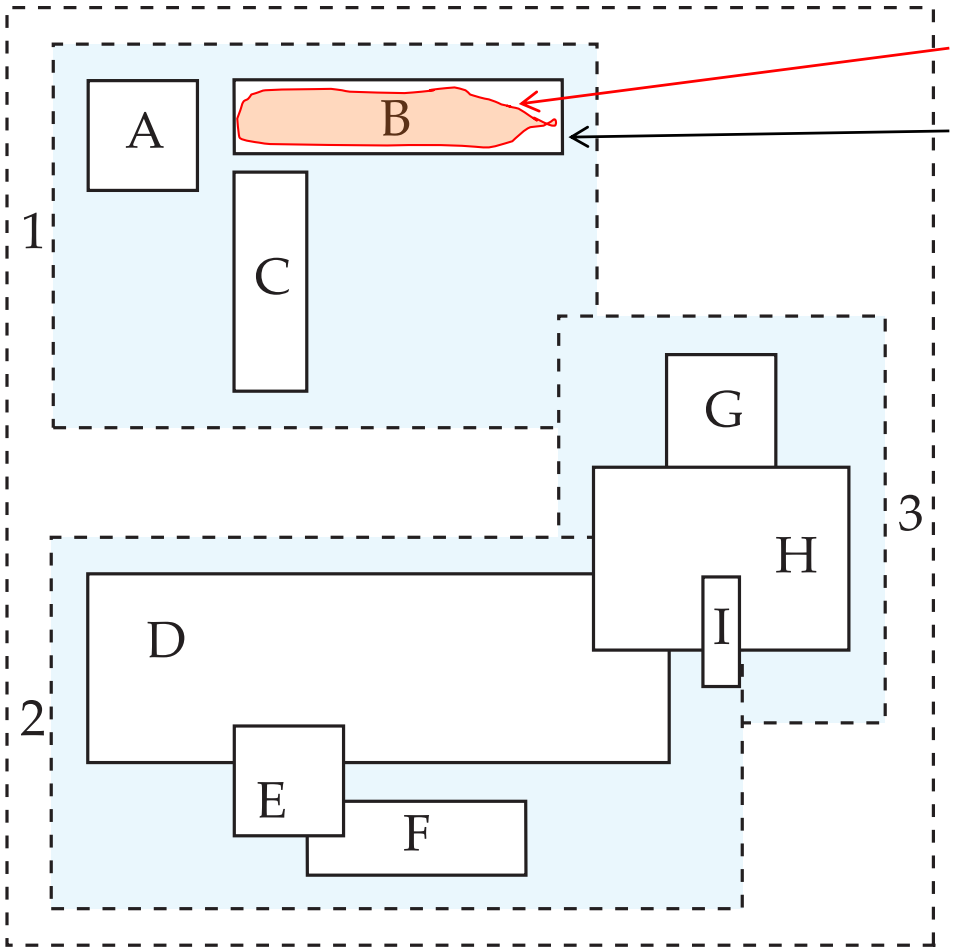
- Search operations: *equality* and *range* predicates in the n dimensional space
  - *Points in a polygon,* e.g., car in my area
  - *Polygons in a polygon,* e.g., house in a flood area
  - *Polygons overlapping with a polygon,* e.g., signal coverage areas
  - *Polygons containing a polygon*

R-tree: A Spatial Index

- Like B-tree:
  - Height-balanced given arbitrary inserts/deletes.
  - Minimum 50% occupancy (except for root).
- Unlike B-tree:
  - There is **no ordered domain** for data entries in the leaf nodes.
    - No total order between n-dimensional points
  - Leaf node: data entry has the form
    <*minimum bounding rectangle MBR, record id rid*>.
  - Non-leaf node: index entry has the form
    <*minimum bounding rectangle MBR, child node pointer p*>.
  - Why rectangles? Use a more precise description?

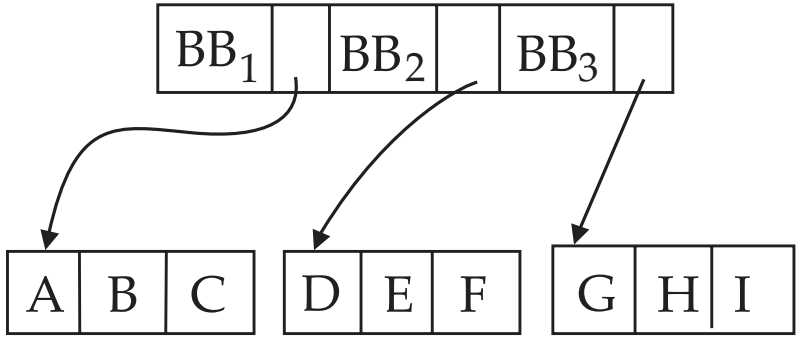
An Example R-tree

Note the containment and overlapping between objects!



Object

Bounding box (or MBR)



Search

- Given a *n-dimensional search rectangle S*, find all data entries that *overlap* with S.
  - At each non-leaf node, find all MBRs overlapping with S, follow their child pointers.
  - At each leaf node, find all MBRs overlapping with S, retrieve the corresponding data records.
  - For each data record, do a detailed comparison.

- Need to traverse multiple paths! (DFS)

R-tree: optimality objectives

- The **area** of a covering rectangle should be minimized
  - Improves performance since decisions on the search paths can be taken high up in the tree
- The **overlap** between covering rectangles should be minimized
  - Decrease the number of paths to be traversed
- The **perimeter** of a covering rectangle should be minimized
  - Favorizes quadratic shapes which can be packed easily
- Storage **utilization** should be optimized

Insertion in an R tree

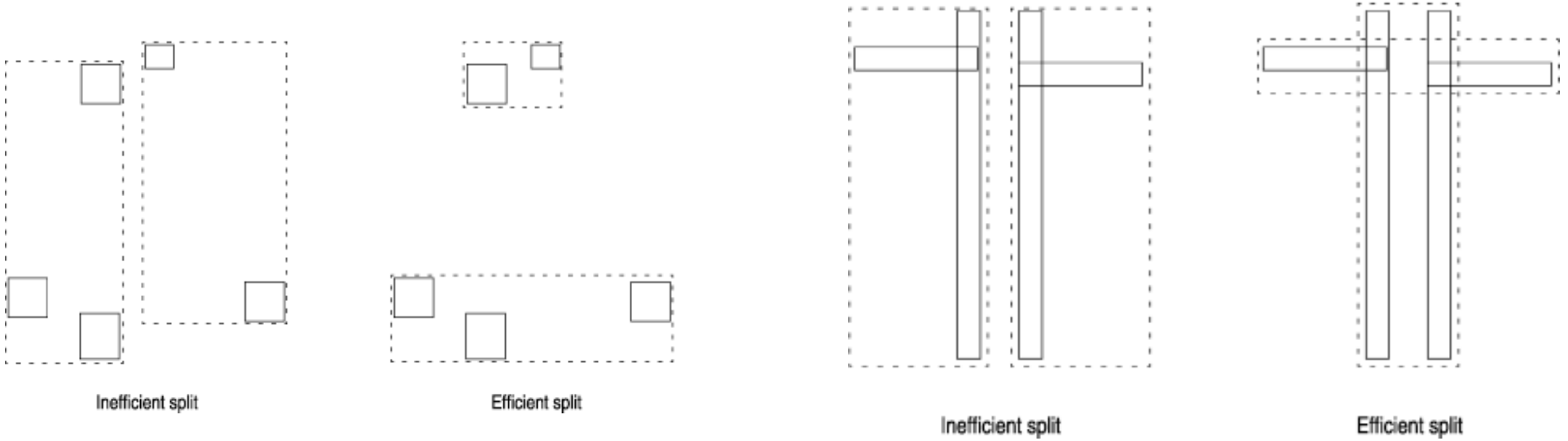
- Insert a data entry **E <*MBR, rid*>**:
- Step 1: Invoke ***ChooseSubtree*** to descend from the root and select a **leaf node L** for insertion.
- Step 2: Insert E to L.
  - If L has room, insert E.
  - Otherwise, invoke ***SplitNode*** to split entries in L and E, into the L node and a **new LL node**.
- Step 3: Invoke ***AdjustTree*** to propagate changes upward, passing L and LL if split.
  - Adjust covering rectangles along the path bottom-up.
  - Propagate node splits as necessary.

ChooseSubtree

- ChooseSubtree: Descend from the root to select a leaf node for placing a new data entry E.
  - Due to MBR overlap, there may be several options!
  - At each level of the tree, pick an index (data) entry leading to the *least enlargement* of the MBR.
    - Break ties by picking a MBR with the *smallest area*.
  - Heuristic choice

SplitNode

- SplitNode: Split M+1 entries (M from the node L, and the new entry E) between L and a new node LL.



  - Exhaustive: enumerate all legal subsets of all M+1 entries, considering the minimum occupancy requirement.
  - Guarantees to find the *optimal* split, but at an *exponential* cost!

SplitNode (Contd.)

- SplitNode: Split entries in the node L plus the new entry E in **two nodes**: L and a new node LL.
  - Quadratic: Sacrifice optimality for performance.
  - *Pick seeds*:
    - Compute the area increase if two entries (Ei, Ej) of L are merged.
    - Pick the pair with *largest area increase*, treat as **the seeds for two groups**.
  - *Split*: Greedily add other entries to the two groups by:
    - picking the next **E'** that *maximizes the difference in area increase between the two groups*,
    - adding **E'** to the group with *smaller area increase* (ties resolved by picking the group with fewer entries).

SplitNode (Contd.)

- SplitNode: Split entries in the node L plus the new entry E between L and a new node LL.
  - Linear split uses a different PickSeeds:
    - For each dimension, pick the entry *with lowest value* and the other *with the highest value*, compute the distance, normalize with the length of the dimension.
    - Among all dimensions, *pick the one with highest normalized distance*.
    - Treat *the two extremes along that dimension* as the **seeds**.

- In practice, some systems (e.g. Postgres) use the 50% minimum occupancy and quadratic split.

Delete

- Remove a data entry E <*MBR*, *rid*>:
- Step 1: Invoke ***FindLeaf*** to descend from the root and locate a leaf node L containing E.
  - Again may traverse multiple paths, but expect just one match.
- Step 2: Delete E from L. Invoke ***CondenseTree***, passing L, to adjust the tree.
  - CondenseTree ensures that the index remains sufficiently full.
  - It may lead to re-inserting some values.
  - Details ommitted.

SplitNode in R-trees*

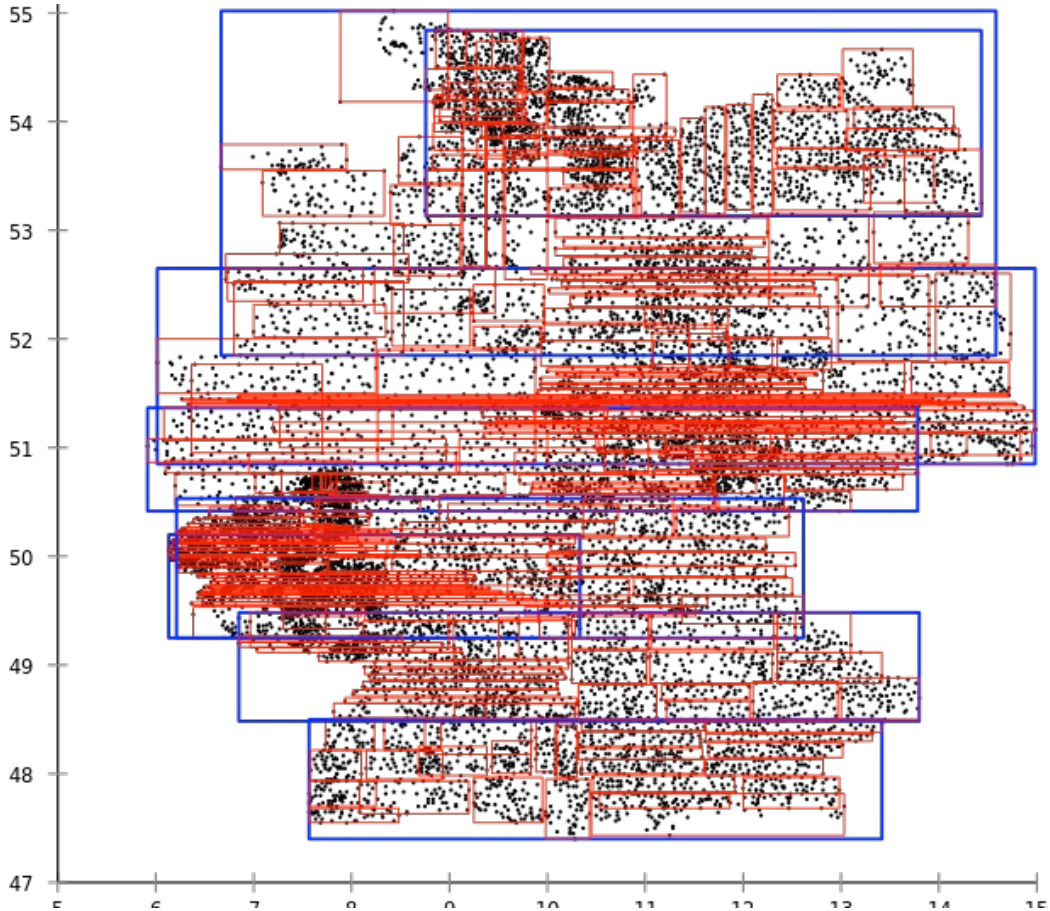
M: maximum nb. of entries, *m*: minimum nb. of entries

1. Along each dimension, sort the *M*+1 entries by the lower value, then by upper value of their rectangles
2. For each sort, create M-2m+2 distributions of the entries
   - *k*-th distribution:
     1. Group 1: the first (m-1)+*k* entries in sort order
     2. Group 2: the remaining entries
3. For each distribution, compute goodness value based on (a) area-value, (b) margin-value, (c) overlap-value.
4. Select the best axis, choose the best distribution:
   - Use (b) to choose split axis,
     (c) to pick a distribution along that axis,
     (a) to break ties.

Works better than R trees *in practice*, on up to 20 dimensions; results depend also on the data.

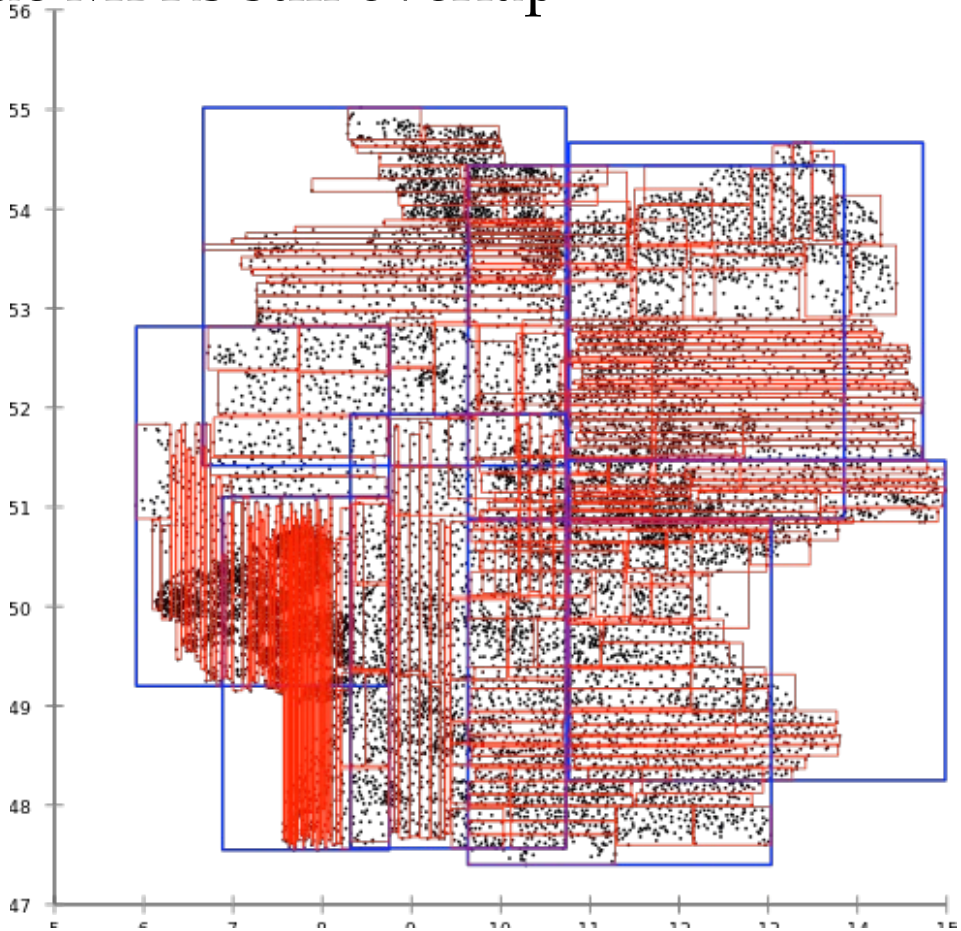
Example: spatial indexing of German zipcodes

- Quadratic split
  - many leaves overlap; index node MBRs overlap, too;
  - some nodes are narrow and occupy all W-E range



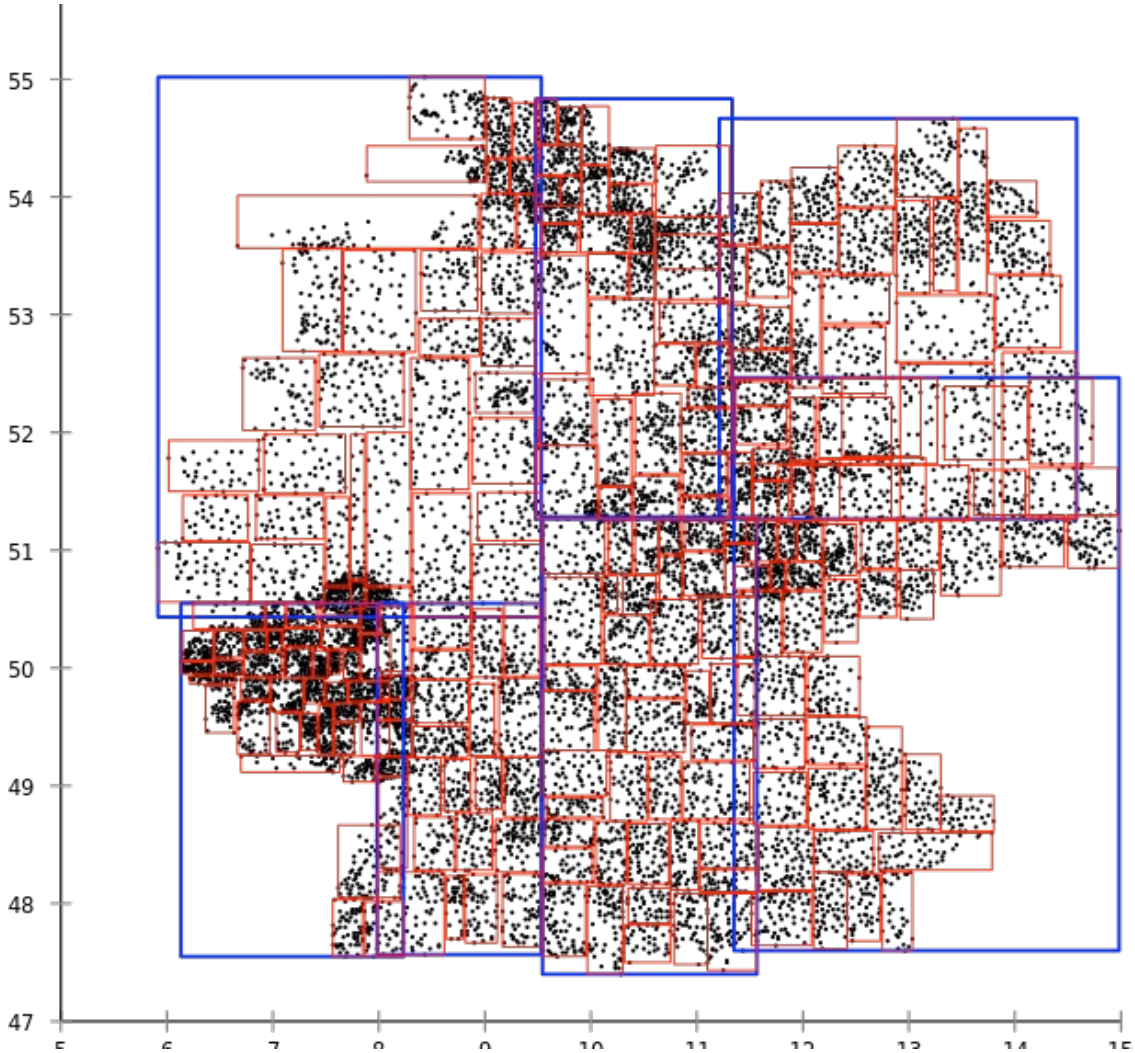
Example: spatial indexing of German zipcodes

- Linear split
  - still many narrow slices; leaf nodes overlap less
  - internal node MBRs still overlap



Example: spatial indexing of German zipcodes

- R*-tree: very good assignment of points in MBRs



Outline

- **Hash-based** indexes
- Indexing for **full-text** search
- R-Tree: **spatial (multidimensional) indexing**
- **Multidimensional indexing** for nearest-neighbor search

Indexing points in high-dimensional spaces

- R-trees are successful for N=2, 3, 4 dimensions
  - Object → N-dimensional bounding box
  - Search: inclusion, overlap, …
- Other applications require much higher N values
  - *And*, object = N-dimensional point (or vector)
    - E.g., N = 10k for k=2, 3,…
    - NLP applications: Word2Vec 100… 1000;
      Sbert: 768 or 1024
  - Search: **nearest neighbor** according to a **distance** (e.g., cosine, Hamming, Euclidean, Jaccard…)
  - No efficient exact methods → approximations:
    trade off (small) precision loss for efficiency
  - Approximate Nearest-Neighbor (**ANN**) search

Indexing high-dimensional vectors

- Popular index data structures include:
  - Hierarchical Navigable Small World Graphs (HNSW)
  - Locality Sensitive Hashing
- Different complexity for:
  - Insert, Search, Update
- Error bounds guaranteed for different (sets of) distances
- Some index libraries are already available for cloud (parallel) deployment
- Among the most popular now: QDrant, FAIS
- Also, extensions in database systems, including Postgres!

ANN within Postgres: pgvector

- Extension to be installed from: https://github.com/pgvector/pgvector

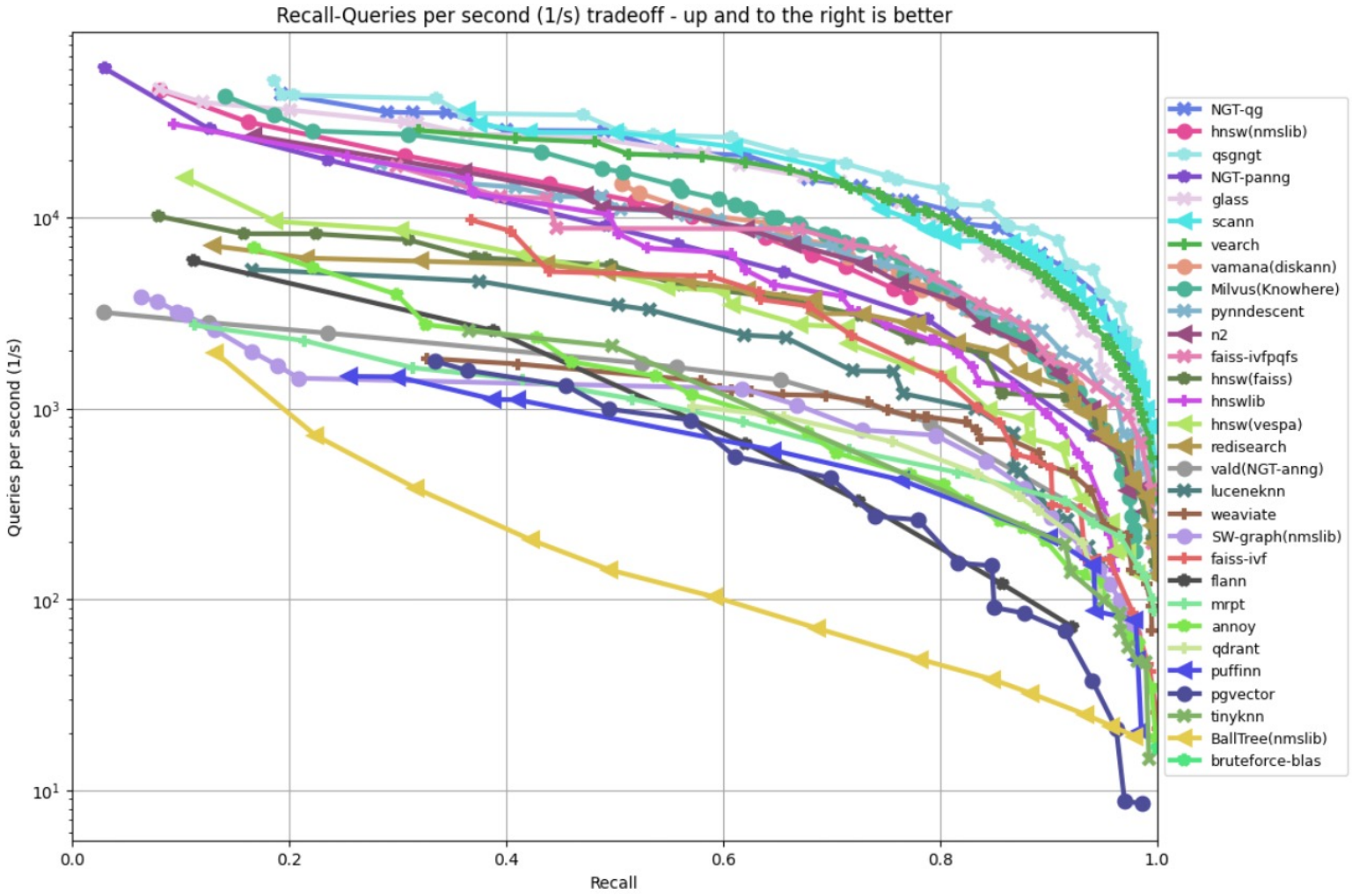
```
CREATE EXTENSION vector;
CREATE TABLE items (id bigserial PRIMARY KEY, embedding vector(3));
INSERT INTO items (embedding) VALUES ('[1,2,3]'), ('[4,5,6]');
-- Get the nearest neighbors by L2 distance:
SELECT * FROM items
ORDER BY embedding <-> '[3,1,2]' LIMIT 5;
```

Also: inner product (<#>), cosine distance (<=>), and L1 distance (<+>)

Sample ANN benchmarks

https://ann-benchmarks.com/index.html

Specific dataset and distance:



Questions

